

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 11 · Hilos y programación concurrente

Creación de hilos: Thread, Runnable y expresiones lambda

Sincronización: synchronized, monitores y visibilidad (volatile)

Coordinación: sleep, join, wait/notify

java.util.concurrent: ExecutorService, Future y pools de hilos

Problemas clásicos: condición de carrera y deadlock

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Concurrencia en Java

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué método pone en marcha un hilo nuevo?

- a) run()
- b) start()
- c) execute()
- d) begin()

RESPUESTA / SOLUCIÓN

b) start() — Crea el hilo del sistema y ejecuta run() en él. Llamar a run() directamente lo ejecuta en el hilo actual.

2. ¿Cuál de estas expresiones crea correctamente un hilo con una lambda?

- a) new Thread(() -> System.out.println("Hola"));
- b) new Thread("Hola");
- c) Thread.start(() -> System.out.println("Hola"));
- d) new Runnable(() -> System.out.println("Hola"));

RESPUESTA / SOLUCIÓN

a) new Thread(() -> ...) — Runnable es una interfaz funcional, por lo que la lambda es válida como argumento del constructor de Thread.

3. Llamar directamente a run() sobre un objeto Thread lanza el código en un hilo nuevo en paralelo.

RESPUESTA / SOLUCIÓN

Falso — `run()` invocado directamente se ejecuta de forma síncrona en el hilo que lo llama. Para ejecutarlo en paralelo hay que usar `start()`.

4. ¿Qué palabra clave impide que dos hilos entren a la vez en un método?

- a) `volatile`
- b) `synchronized`
- c) `locked`
- d) `atomic`

RESPUESTA / SOLUCIÓN

b) `synchronized` — Serializa el acceso al método/bloque mediante el monitor del objeto.

5. Si un hilo llama a `wait()` sin haber adquirido el monitor (sin estar en un bloque `synchronized`), ¿qué ocurre?

- a) Se bloquea hasta que otro hilo la notifique
- b) Se lanza `IllegalMonitorStateException`
- c) Se lanza `InterruptedException`
- d) No ocurre nada

RESPUESTA / SOLUCIÓN

b) `IllegalMonitorStateException` — `wait()` solo es legal si el hilo posee el monitor del objeto.

-
6. **`volatile` garantiza la visibilidad de una variable entre hilos, pero no la atomicidad de operaciones compuestas como `contador++`.**

RESPUESTA / SOLUCIÓN

Verdadero — `volatile` evita que cada hilo use una copia en caché, pero no serializa lecturas-modificaciones-escrituras; para eso se necesita `synchronized` o clases atómicas.

-
7. **¿Qué interfaz de `java.util.concurrent` permite ejecutar tareas en un pool de hilos?**
- a) `ThreadPool`
 - b) `ExecutorService`
 - c) `RunnableService`
 - d) `FuturePool`

RESPUESTA / SOLUCIÓN

b) `ExecutorService` — Gestiona un pool de hilos y ofrece `submit()` para lanzar tareas y obtener `Future`.

8. ¿Qué método de Thread hace que el hilo actual espere a que termine otro?

- a) sleep()
- b) yield()
- c) join()
- d) wait()

RESPUESTA / SOLUCIÓN

c) join() — El hilo que lo invoca espera a que el hilo destino finalice.

9. Un deadlock ocurre cuando dos o más hilos esperan, cada uno, un recurso que otro mantiene, bloqueándose indefinidamente.

RESPUESTA / SOLUCIÓN

Verdadero — Es una espera circular: A espera el recurso de B y B espera el de A. Se evita, por ejemplo, adquiriendo los recursos siempre en el mismo orden.

10. ¿Qué excepción hay que capturar o declarar al usar Thread.sleep()?

- a) IOException
- b) InterruptedException
- c) RuntimeException
- d) SleepException

RESPUESTA / SOLUCIÓN

b) InterruptedException — sleep() la lanza si otro hilo interrumpe al que duerme.

Parte 2: Análisis y depuración

3 puntos

1. **Analiza el siguiente código. ¿Qué valor imprime normalmente? ¿Puede imprimir algo distinto? Explica por qué. (1,5 pts)**

```
public class Carrera {
    static int contador = 0;
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(() -> { for (int i = 0; i < 1000; i++) contador++; });
        Thread t2 = new Thread(() -> { for (int i = 0; i < 1000; i++) contador++; });
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println(contador);
    }
}
```

RESPUESTA / SOLUCIÓN

contador++ no es atómico: son tres operaciones (leer, sumar, escribir). Dos hilos pueden leer el mismo valor y escribir el mismo resultado, perdiendo incrementos.

El valor esperado sería 2000, pero en la práctica suele salir menos (p. ej. 1923) y es distinto en cada ejecución: es una condición de carrera.

Solución: declarar el método con synchronized (o usar AtomicInteger): public static synchronized void inc() { contador++; }.

-
2. **El siguiente código a veces se bloquea sin terminar. ¿Qué problema tiene y cómo se corrige? (1,5 pts)**

```
public class Bloqueo {
    static final Object A = new Object();
    static final Object B = new Object();
    public static void main(String[] args) {
        new Thread(() -> { synchronized (A) {
            try { Thread.sleep(50); } catch (InterruptedException e) {}
            synchronized (B) { System.out.println("Hilo 1"); }
        } }).start();
        new Thread(() -> { synchronized (B) {
            try { Thread.sleep(50); } catch (InterruptedException e) {}
            synchronized (A) { System.out.println("Hilo 2"); }
        } }).start();
    }
}
```

RESPUESTA / SOLUCIÓN

Es un deadlock clásico: el hilo 1 toma A y espera B, mientras el hilo 2 toma B y espera A. Ninguno libera su recurso y ambos quedan bloqueados para siempre.

Corrección: adquirir los bloqueos siempre en el mismo orden (p. ej. primero A y después B en los dos hilos), o usar tryLock con `java.util.concurrent.locks.Lock`.

Parte 3: Ejercicios de programación

6 puntos

1. Escribe un programa que cree 5 hilos (Runnable con lambda). Cada hilo imprime su número y una iteración 3 veces, duerme 100 ms entre impresiones, y el main espera a todos con join() antes de imprimir el mensaje final. (2 pts)

RESPUESTA / SOLUCIÓN

```
public class HilosBasicos {  
    public static void main(String[] args) throws InterruptedException {  
        Thread[] hilos = new Thread[5];  
        for (int i = 1; i <= 5; i++) {  
            final int num = i;  
            hilos[i - 1] = new Thread(() -> {  
                for (int j = 1; j <= 3; j++) {  
                    System.out.println("Hilo " + num + ": iteración " + j);  
                    try { Thread.sleep(100); } catch (InterruptedException e) { }  
                }  
            });  
            hilos[i - 1].start();  
        }  
        for (Thread h : hilos) h.join();  
        System.out.println("Todos los hilos han terminado");  
    }  
}
```

2. Implementa una clase Contador con un método incrementar() sincronizado y un main que lance 10 hilos que incrementen 1000 veces cada uno. Comprueba que el resultado final es exactamente 10000. (2 pts)

RESPUESTA / SOLUCIÓN

```
public class ContadorSeguro {  
    private int valor = 0;  
    public synchronized void incrementar() { valor++; }  
    public synchronized int getValor() { return valor; }  
    public static void main(String[] args) throws InterruptedException {  
        ContadorSeguro c = new ContadorSeguro();  
        Thread[] hilos = new Thread[10];  
        for (int i = 0; i < 10; i++) {  
            hilos[i] = new Thread(() -> {  
                for (int j = 0; j < 1000; j++) c.incrementar();  
            });  
            hilos[i].start();  
        }  
        for (Thread h : hilos) h.join();  
  
        System.out.println("Valor final: " + c.getValor()); // 10000  
    }  
}
```

3. Usa `ExecutorService` con un pool fijo de 3 hilos para calcular el cuadrado de los números 1 a 6. Cada tarea debe ser un `Callable<Integer>` y el main debe mostrar los resultados con `Future.get()`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class PoolTareas {

    public static void main(String[] args) throws Exception {

        ExecutorService pool = Executors.newFixedThreadPool(3);

        List<Future<Integer>> futuros = new ArrayList<>();

        for (int n = 1; n <= 6; n++) {

            final int v = n;

            futuros.add(pool.submit(() -> v * v));

        }

        for (Future<Integer> f : futuros) {

            System.out.println("Cuadrado: " + f.get());

        }

        pool.shutdown();

    }

}
```

Parte 4: Pregunta teórica

2 puntos

1. Explica la diferencia entre `sleep()` y `wait()`, y entre heredar de `Thread` e implementar `Runnable`. ¿Cuándo conviene usar `ExecutorService`?

RESPUESTA / SOLUCIÓN

`sleep()` es estático y solo pausa el hilo actual sin liberar el monitor; `wait()` libera el monitor del objeto y debe llamarse desde un bloque `synchronized`, quedando el hilo a la espera de `notify()/notifyAll()`.

Implementar `Runnable` es preferible a heredar `Thread` porque no se gasta la única herencia de clase y separa la tarea del mecanismo de ejecución.

`ExecutorService` conviene cuando hay muchas tareas cortas o repetidas: reutiliza hilos (evita el coste de crearlos), limita el número máximo con el pool y permite obtener resultados con `Future`.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)